Laboratorio Técnico: Arquitectura de Estado Reactivo y Despliegue Móvil

Módulo: Desarrollo de Interfaces (0488) | **Ciclo:** DAM (Grado Superior)
Alineación Curricular:

- **RA 1 (CE 1.a, CE 1.b):** Aplicación de patrones de diseño y arquitecturas (MVVM, Store, SOLID).
- **RA 3 (CE 3.d):** Gestión de eventos y enlace de datos (Data-binding reactivo).
- **RA 4 (CE 4.b):** Usabilidad y retroalimentación de estado asíncrono.
- **RA 7 (CE 7.a, 7.c):** Preparación, empaquetado y distribución de la aplicación (Android).

1. Comandos de Inicialización y Andamiaje (CLI)

Para comenzar el desarrollo de "TechTicket FP", abriremos la terminal y ejecutaremos la siguiente secuencia de comandos. Esto generará la estructura base asegurando un entorno Standalone (sin NgModules).

```
Bash
# 1. Crear el proyecto base (Angular Standalone)
ionic start tech-ticket blank --type=angular --standalone

# 2. Entrar al directorio
cd tech-ticket

# 3. Generar el servicio que actuará como Store de estado
ionic generate service core/services/ticket

# 4. Generar el componente visual del panel de control
ionic generate component features/ticket-dashboard --standalone
```

2. Decisiones Arquitectónicas: Patrones y Principios SOLID

Antes de codificar, es crítico justificar la arquitectura de software aplicada en este laboratorio (evaluable en RA 1).

Patrón de Diseño: State Management / Store Pattern (Variante MVVM)

Utilizaremos un patrón de gestión de estado centralizado impulsado por Angular Signals. El Servicio actúa como el *Model/ViewModel* que retiene la "fuente única de la verdad" (Single Source of Truth), mientras que el Componente es puramente una *Vista* reactiva.

- **¿Por qué?:** Desacopla la lógica de acceso a datos (HTTP) y la gestión del estado asíncrono (cargando, error, datos) de la interfaz gráfica. Al usar Signals, permitimos que el framework actualice el DOM de forma granular (Zoneless) sin recálculos innecesarios.

Aplicación de Principios SOLID

- **SRP (Principio de Responsabilidad Única):** El TicketService tiene la única responsabilidad de mutar y mantener el estado de los tickets. El TicketDashboardComponent tiene la única responsabilidad de renderizar los datos y capturar los eventos del usuario (clicks, filtros).
- **OCP (Principio de Abierto/Cerrado):** El estado del servicio se expone mediante señales derivadas (computed). La vista está abierta a la extensión (podemos añadir nuevos selectores) pero el estado base está cerrado a la modificación directa desde fuera (encapsulamiento).
- **DIP (Principio de Inversión de Dependencias):** Se aplica mediante el sistema de inyección de dependencias de Angular (inject(HttpClient)). El componente no instancia el servicio, sino que Angular se lo provee, facilitando futuros tests unitarios con *mocks*.

3. Implementación del Laboratorio

Fase A: El Servicio de Estado (src/app/core/services/ticket.service.ts)

Implementamos el patrón Store encapsulando el estado mutable de la API.

```
TypeScript
import { Injectable, signal, computed, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';

export interface Ticket {
  id: number;
  title: string;
  completed: boolean;
}

interface TicketState {
  data: Ticket[];
  loading: boolean;
  error: string | null;
}

@Injectable({ providedIn: 'root' })
export class TicketService {
  // DIP: Inyección de dependencias moderna
  private http = inject(HttpClient);

  // OCP & Encapsulación: El estado es privado y mutable solo desde aquí
  private state = signal<TicketState>({
    data: [],
    loading: false,
    error: null
  });

  // Exposición controlada (Solo lectura / Derivaciones reactivas)
  public tickets = computed(() => this.state().data);
  public isLoading = computed(() => this.state().loading);
  public errorMessage = computed(() => this.state().error);

  public fetchTickets(): void {
    // Mutación inicial: Activamos el indicador de carga
    this.state.update(s => ({ ...s, loading: true, error: null }));

    this.http.get<Ticket[]>('https://jsonplaceholder.typicode.com/todos?_limit=15')
```

```
      .subscribe({
        next: (response) => {
          this.state.update(s => ({ ...s, data: response, loading: false }));
        },
        error: () => {
          this.state.update(s => ({
            ...s,
            error: 'Error de red al contactar con el servidor. Revise su conexión.',
            loading: false
          }));
        }
      });
  }
}
```

Fase B: El Componente Visual (src/app/features/ticket-dashboard/ticket-dashboard.component.ts)

Conectamos la UI al servicio y aplicamos una señal local para el filtrado en memoria.

```
TypeScript
import { Component, computed, inject, OnInit, signal, ChangeDetectionStrategy } from
'@angular/core';
import { TicketService } from '../../core/services/ticket.service';
import { IonHeader, IonToolbar, IonTitle, IonContent, IonSegment, IonSegmentButton,
IonLabel, IonSpinner, IonText, IonButton, IonList, IonItem, IonBadge } from
'@ionic/angular/standalone';

@Component({
 selector: 'app-ticket-dashboard',
 standalone: true,
 changeDetection: ChangeDetectionStrategy.OnPush, // Optimización de renderizado
 imports: [IonHeader, IonToolbar, IonTitle, IonContent, IonSegment, IonSegmentButton,
IonLabel, IonSpinner, IonText, IonButton, IonList, IonItem, IonBadge],
 template: `
  <ion-header>
   <ion-toolbar color="dark">
    <ion-title>TechTicket FP</ion-title>
   </ion-toolbar>
  </ion-header>

  <ion-content class="ion-padding">
   <!-- UI: Selector de Filtro -->
   <ion-segment (ionChange)="cambiarFiltro($event)" value="todos" class="ion-margin-bottom">
    <ion-segment-button value="todos"><ion-label>Todos</ion-label></ion-segment-button>
    <ion-segment-button value="abiertos"><ion-label>Abiertos</ion-label></ion-segment-button>
    <ion-segment-button
value="cerrados"><ion-label>Cerrados</ion-label></ion-segment-button>
   </ion-segment>

   <!-- Control Flow: Gestión de Estado Asíncrono (Feedback UI) -->
   @if (ticketService.isLoading()) {
    <div class="ion-text-center ion-margin-top">
     <ion-spinner name="dots"></ion-spinner>
     <p>Sincronizando incidencias...</p>
    </div>
   } @else if (ticketService.errorMessage()) {
    <div class="ion-text-center">
     <ion-text color="danger">
      <p>{{ ticketService.errorMessage() }}</p>
     </ion-text>
```

```
        <ion-button (click)="cargarDatos()" size="small">Reintentar</ion-button>
      </div>
    } @else {
      <!-- Renderizado Optimizado de la Lista -->
      <ion-list>
        @for (ticket of ticketsFiltrados(); track ticket.id) {
          <ion-item>
            <ion-label>
              <h2>#{{ ticket.id }} - {{ ticket.title }}</h2>
            </ion-label>
            <ion-badge slot="end" [color]="ticket.completed ? 'success' : 'warning'">
              {{ ticket.completed ? 'Resuelto' : 'Pendiente' }}
            </ion-badge>
          </ion-item>
        } @empty {
          <ion-item><ion-label class="ion-text-center">No hay incidencias en esta
vista.</ion-label></ion-item>
        }
      </ion-list>
    }
  </ion-content>
  `
})
export class TicketDashboardComponent implements OnInit {
  public ticketService = inject(TicketService);

  // Estado local de la vista
  public filtroActual = signal<'todos' | 'abiertos' | 'cerrados'>('todos');

  // Composición Reactiva: Se recalcula automáticamente si cambia la API o el filtro
  public ticketsFiltrados = computed(() => {
    const data = this.ticketService.tickets();
    const filtro = this.filtroActual();

    if (filtro === 'abiertos') return data.filter(t => !t.completed);
    if (filtro === 'cerrados') return data.filter(t => t.completed);
    return data;
  });

  ngOnInit() {
    this.cargarDatos();
  }

  public cargarDatos(): void {
    this.ticketService.fetchTickets();
```

```
  }

  public cambiarFiltro(event: any): void {
    this.filtroActual.set(event.detail.value);
  }
}
```

4. Empaquetado y Despliegue en Android (Capacitor)

Para cumplir con el **RA 7**, trasladaremos nuestra aplicación web reactiva a un contenedor nativo Android utilizando Capacitor. Este proceso compila el código TypeScript/Angular en un *bundle* estático (HTML/JS/CSS) y lo inyecta en un proyecto nativo de Android Studio.
Ejecuta la siguiente secuencia técnica en la terminal, en la raíz del proyecto:

1. **Compilar el proyecto de producción:**
   Genera los archivos optimizados (dist/) de la aplicación Angular.
   Bash
   ionic build --prod

2. **Integrar Capacitor y la plataforma Android:**
   Añade el andamiaje nativo al proyecto (esto creará la carpeta android/ en la raíz).
   Bash
   npx cap add android

3. **Sincronizar el código compilado:**
   Copia los archivos del *build* web generados en el paso 1 dentro del proyecto nativo de Android. Este comando se debe ejecutar **cada vez** que realices un cambio en el código Angular y vuelvas a hacer ionic build.
   Bash
   npx cap sync android

4. **Ejecución y Testing en el dispositivo:**
   Tienes dos vías para visualizar la aplicación:
   - **Vía IDE (Recomendado para ajustes nativos):** Abre Android Studio con el proyecto preconfigurado.
     Bash
     npx cap open android
   - **Vía CLI (Despliegue directo al emulador o dispositivo físico conectado por USB/ADB):**
     Bash
     npx cap run android